Instituto Tecnológico de Las Américas (ITLA)

Programación-Paralela



Práctica de Laboratorio 1

Configuración del Entorno y Hola Mundo Multihilo en C#

Estudiante: Brian Amaury Guzman Mañon

Matrícula: 2024-1286

Docente: Erick Leonardo Perez Veloz

Repositorio: https://github.com/Printbriandev/Practica-1-p-paralela

Índice

1. Introducción 2
2. Objetivos 2
2.1 Objetivo general 2
2.2 Objetivos específicos 2
3. Marco teórico 2
3.1 Concurrencia y paralelismo 2
3.2 Procesadores lógicos y Environment.ProcessorCount 2
3.3 Task Parallel Library (TPL) y ThreadPool 2
3.4 Parallel.For y ParallelOptions.MaxDegreeOfParallelism 3
3.5 Task.Run, Task.WhenAll y SemaphoreSlim 3
3.6 Stopwatch y speedup 3
4. Configuración del entorno 3
Figura 1. Solución HolaMundoMultihilo abierta en Visual Studio 2026 (Program.cs). 4
5. Desarrollo 4
5.1 Detección de procesadores y configuración del paralelismo 4
5.2 Fase 1: Ejecución secuencial 4
5.3 Fase 2: Parallel.For 4
5.4 Fase 3: Task.Run con SemaphoreSlim 5
5.5 Verificación del límite 5
6. Código fuente 5
7. Resultados 8
Figura 2. Detección de procesadores lógicos y configuración de MaxDegreeOfParallelism. 8
..... 9
Figura 3. Ejecución de la fase Parallel.For: nunca hay más de 12 tareas simultáneas y los hilos se reutilizan. 9
..... 9
Figura 4. Resumen de tiempos de ejecución, speedup, concurrencia máxima e hilos utilizados. 9
7.1 Análisis 9
8. Conclusiones 10
9. Repositorio 10
10. Referencias 10

1. Introducción

Los procesadores actuales incluyen varios núcleos físicos y, gracias a tecnologías como Hyper-Threading, exponen aún más procesadores lógicos al sistema operativo. Un programa tradicional, escrito de forma secuencial, solo aprovecha uno de ellos a la vez, dejando el resto del hardware inactivo.

En esta práctica se configura el entorno de desarrollo y se construye una aplicación de consola en C# que detecta en tiempo de ejecución cuántos procesadores lógicos tiene el equipo y utiliza ese valor para controlar cuántas tareas se ejecutan en paralelo. Además, se compara el tiempo de la ejecución secuencial con dos estrategias paralelas de la Task Parallel Library (TPL): Parallel.For y Task.Run.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación de consola en C# que lance tareas en paralelo controlando explícitamente el grado máximo de paralelismo a partir de la cantidad de procesadores lógicos del sistema.

2.2 Objetivos específicos

- Instalar y configurar Visual Studio y el SDK de .NET para compilar aplicaciones de consola en C#.
- Obtener la cantidad de procesadores lógicos mediante `Environment.ProcessorCount`.
- Configurar `MaxDegreeOfParallelism` de forma dinámica con ese valor.
- Comparar los tiempos de ejecución secuencial y paralela, y calcular el speedup obtenido.
- Verificar que el número de tareas simultáneas nunca supera el límite configurado.

3. Marco teórico

3.1 Concurrencia y paralelismo

La concurrencia consiste en tener varias tareas en progreso durante el mismo período; el paralelismo es el caso particular en que esas tareas se ejecutan literalmente al mismo tiempo en distintos procesadores. El modelo utilizado en esta práctica es el de bifurcación-unión (fork-join): el programa divide el trabajo en varias tareas (fork), las ejecuta en paralelo y espera a que todas terminen antes de continuar (join).

3.2 Procesadores lógicos y Environment.ProcessorCount

`Environment.ProcessorCount` es una propiedad estática de .NET que devuelve el número de procesadores lógicos disponibles para el proceso actual, incluyendo los hilos de hardware que aporta Hyper-Threading o SMT. Consultarla en tiempo de ejecución permite que el mismo programa se adapte a cualquier equipo sin modificar el código.

3.3 Task Parallel Library (TPL) y ThreadPool

La TPL es el conjunto de tipos de .NET (System.Threading.Tasks) que simplifica la programación paralela. En lugar de crear hilos manualmente, el programador define tareas y la TPL las distribuye entre los hilos del ThreadPool, un grupo de hilos reutilizables que el entorno de ejecución administra de forma dinámica: crea hilos cuando hacen falta y reutiliza los que ya terminaron su trabajo.

Por este motivo no se debe forzar un mapeo estático de un hilo por núcleo. El sistema operativo decide en qué núcleo corre cada hilo, y el planificador de la TPL decide qué hilo ejecuta cada tarea. El programador solo define el límite de tareas simultáneas.

3.4 Parallel.For y ParallelOptions.MaxDegreeOfParallelism

`Parallel.For` ejecuta las iteraciones de un bucle en paralelo, particionándolas automáticamente entre hilos del ThreadPool. Recibe un objeto `ParallelOptions` cuya propiedad `MaxDegreeOfParallelism` establece el número máximo de iteraciones que pueden ejecutarse al mismo tiempo. Es un límite superior, no una asignación fija de hilos.

3.5 Task.Run, Task.WhenAll y SemaphoreSlim

`Task.Run` encola un trabajo en el ThreadPool y devuelve un objeto `Task` que lo representa. `Task.WhenAll` espera a que un conjunto de tareas termine, y actúa como punto de unión. Como `Task.Run` no acepta `ParallelOptions`, el límite de concurrencia se aplica con un `SemaphoreSlim`: un semáforo con N permisos que solo deja pasar a N tareas a la vez; las demás esperan hasta que alguna libera su permiso.

3.6 Stopwatch y speedup

`Stopwatch` (System.Diagnostics) mide intervalos de tiempo con alta precisión. A partir de los tiempos medidos se calcula el speedup, que indica cuántas veces más rápida es la versión paralela:

Speedup = Tiempo secuencial / Tiempo paralelo

4. Configuración del entorno

Para el desarrollo se utilizó el siguiente entorno:

Componente	Detalle
Sistema operativo	Windows 11 Home (10.0.26200)
IDE	Visual Studio Community 2026 (versión 18)
SDK de .NET	.NET SDK 10.0.401 (runtime 10.0.12)
Lenguaje	C# (proyecto de consola, net10.0)
Procesadores lógicos	12
Control de versiones	Git + GitHub

Pasos realizados:

1. Instalación de Visual Studio 2026 con la carga de trabajo «Desarrollo de escritorio de .NET», que incluye el SDK de .NET.
2. Verificación del SDK desde la terminal con `dotnet --list-sdks`.
3. Creación de la solución `HolaMundoMultihilo.slnx` y del proyecto de consola `HolaMundoMultihilo`.
4. Implementación del programa en Program.cs, compilación y ejecución con Ctrl+F5.
5. Publicación del código fuente en el repositorio de GitHub.

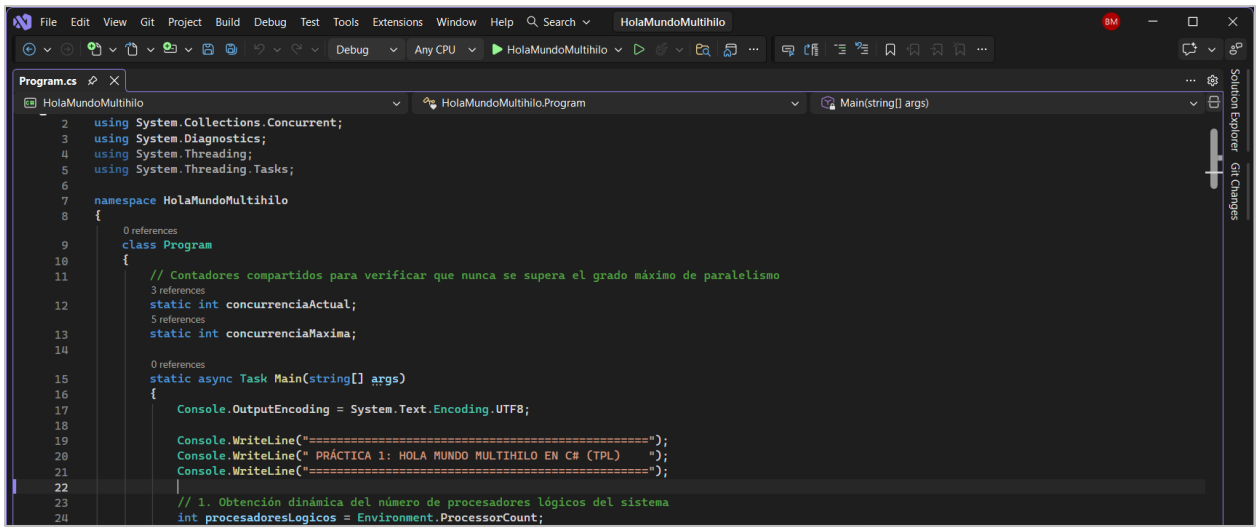


Figura 1. Solución HolaMundoMultihilo abierta en Visual Studio 2026 (Program.cs).

5. Desarrollo

El programa ejecuta la misma carga de trabajo con tres estrategias y mide el tiempo de cada una. Cada tarea imprime un mensaje «Hola Mundo» indicando su número, el identificador del hilo que la ejecuta, si ese hilo pertenece al ThreadPool y cuántas tareas se están ejecutando simultáneamente. Luego simula trabajo con una pausa de 300 ms.

5.1 Detección de procesadores y configuración del paralelismo

Al iniciar, el programa lee `Environment.ProcessorCount` y crea un objeto `ParallelOptions` con `MaxDegreeOfParallelism` igual a ese valor. Se lanzan el doble de tareas que procesadores (24 en este equipo) para que el efecto del límite sea visible: si hubiera igual o menos tareas que procesadores, el límite nunca llegaría a actuar.

5.2 Fase 1: Ejecución secuencial

Un bucle `for` ejecuta las tareas una detrás de otra en el hilo principal. Sirve como línea base para comparar.

5.3 Fase 2: Parallel.For

`Parallel.For` recibe las opciones con el límite configurado. La TPL reparte las 24 iteraciones entre hilos del ThreadPool, sin permitir más de 12 simultáneas. El hilo principal también participa en el trabajo.

5.4 Fase 3: Task.Run con SemaphoreSlim

Se crean 24 tareas con `Task.Run`. Cada una debe obtener un permiso del `SemaphoreSlim` (inicializado con `Environment.ProcessorCount`) antes de trabajar, y lo libera al terminar. `Task.WhenAll` espera a que todas finalicen.

5.5 Verificación del límite

Para comprobar que el límite se respeta, cada tarea incrementa un contador compartido al empezar y lo decrementa al terminar, usando operaciones atómicas de la clase `Interlocked` para evitar condiciones de carrera. El valor más alto alcanzado por ese contador es la concurrencia máxima observada, que se muestra en el resumen final.

6. Código fuente

A continuación se presenta el contenido completo de `Program.cs`:

```
using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Threading;
using System.Threading.Tasks;

namespace HolaMundoMultihilo
{
    class Program
    {
        // Contadores compartidos para verificar que nunca se supera el grado máximo de paralelismo
        static int concurrenciaActual;
        static int concurrenciaMaxima;

        static async Task Main(string[] args)
        {
            Console.OutputEncoding = System.Text.Encoding.UTF8;

            Console.WriteLine("================================================");
            Console.WriteLine(" PRÁCTICA 1: HOLA MUNDO MULTIHILO EN C# (TPL)    ");
            Console.WriteLine("================================================");

            // 1. Obtención dinámica del número de procesadores lógicos del sistema
            int procesadoresLogicos = Environment.ProcessorCount;
            Console.WriteLine($"\n[INFO] Sistema operativo            : {Environment.OSVersion}");
            Console.WriteLine($"[INFO] Versión de .NET              : {Environment.Version}");
            Console.WriteLine($"[INFO] Procesadores lógicos         : {procesadoresLogicos}");

            // 2. Grado máximo de paralelismo configurado a partir de Environment.ProcessorCount.
            //    Solo se limita cuántas tareas corren a la vez; el planificador de la TPL decide
            //    qué hilo del ThreadPool ejecuta cada una (no hay mapeo fijo hilo-núcleo).
            var opciones = new ParallelOptions
            {
                MaxDegreeOfParallelism = procesadoresLogicos
            };
            Console.WriteLine($"[INFO] MaxDegreeOfParallelism       : {opciones.MaxDegreeOfParallelism}");
```

```
            // Se lanzan más tareas que procesadores para que el límite sea observable
            int totalTareas = procesadoresLogicos * 2;
            const int duracionSimuladaMs = 300; // Simulación de carga de trabajo
            Console.WriteLine($"[INFO] Tareas por fase             : {totalTareas}");

            // ------------------------------------------------------------
            // FASE 1: Ejecución Secuencial (bucle tradicional for)
            // ------------------------------------------------------------
            ImprimirEncabezado("1. Iniciando Ejecución Secuencial...");

            Stopwatch swSecuencial = Stopwatch.StartNew();
            for (int i = 0; i < totalTareas; i++)
            {
                ProcesarTarea(i, "SECUENCIAL", duracionSimuladaMs);
            }
            swSecuencial.Stop();
            Console.WriteLine($"[RESULTADO] Tiempo total secuencial: {swSecuencial.ElapsedMilliseconds}
ms");

            // ------------------------------------------------------------
            // FASE 2: Parallel.For con MaxDegreeOfParallelism explícito
            // ------------------------------------------------------------
            ImprimirEncabezado("2. Iniciando Ejecución Paralela con Parallel.For...");
            ReiniciarContadores();

            var hilosParallelFor = new ConcurrentDictionary<int, int>();
            Stopwatch swParallelFor = Stopwatch.StartNew();
            Parallel.For(0, totalTareas, opciones, index =>
            {
                hilosParallelFor.AddOrUpdate(Environment.CurrentManagedThreadId, 1, (_, n) => n + 1);
                ProcesarTarea(index, "PARALLEL.FOR", duracionSimuladaMs);
            });
            swParallelFor.Stop();
            int maxParallelFor = concurrenciaMaxima;
            Console.WriteLine($"[RESULTADO] Tiempo total con Parallel.For:
{swParallelFor.ElapsedMilliseconds} ms");

            // ------------------------------------------------------------
            // FASE 3: Task.Run + Task.WhenAll limitado con SemaphoreSlim
            // ------------------------------------------------------------
            // Task.Run no acepta ParallelOptions, así que el mismo límite se aplica con un
            // SemaphoreSlim inicializado con Environment.ProcessorCount.
            ImprimirEncabezado("3. Iniciando Ejecución Asíncrona con Task.Run...");
            ReiniciarContadores();

            var hilosTaskRun = new ConcurrentDictionary<int, int>();
            using var limitador = new SemaphoreSlim(opciones.MaxDegreeOfParallelism);
            Stopwatch swTaskRun = Stopwatch.StartNew();
            Task[] arregloTareas = new Task[totalTareas];

            for (int i = 0; i < totalTareas; i++)
            {
                int taskId = i; // Captura local para evitar problemas de cierre (closure)
                arregloTareas[i] = Task.Run(async () =>
                {
                    await limitador.WaitAsync();
                    try
                    {
                        hilosTaskRun.AddOrUpdate(Environment.CurrentManagedThreadId, 1, (_, n) => n + 1);
                        ProcesarTarea(taskId, "TASK.RUN", duracionSimuladaMs);
                    }
                    finally
                    {
                        limitador.Release();
```

```
                }
            });
        }

        // Espera la finalización de todas las tareas iniciadas (punto de unión / join)
        await Task.WhenAll(arregloTareas);
        swTaskRun.Stop();
        int maxTaskRun = concurrenciaMaxima;
        Console.WriteLine($"[RESULTADO] Tiempo total con Task.Run: {swTaskRun.ElapsedMilliseconds}
ms");

        // ------------------------------------------------------------
        // RESUMEN DE COMPARACIÓN Y RENDIMIENTO
        // ------------------------------------------------------------
        double speedupFor = (double)swSecuencial.ElapsedMilliseconds /
swParallelFor.ElapsedMilliseconds;
        double speedupTask = (double)swSecuencial.ElapsedMilliseconds / swTaskRun.ElapsedMilliseconds;

        Console.WriteLine("\n================================================");
        Console.WriteLine(" RESUMEN DE TIEMPOS DE EJECUCIÓN ");
        Console.WriteLine("================================================");
        Console.WriteLine($"Procesadores lógicos    : {procesadoresLogicos}");
        Console.WriteLine($"MaxDegreeOfParallelism  : {opciones.MaxDegreeOfParallelism}");
        Console.WriteLine("------------------------------------------------");
        Console.WriteLine("Estrategia    | Tiempo (ms) | Speedup | Conc. máx | Hilos");
        Console.WriteLine($"Secuencial    | {swSecuencial.ElapsedMilliseconds,11} | {1.0,6:F2}x |
{1,9} | {1,5}");
        Console.WriteLine($"Parallel.For  | {swParallelFor.ElapsedMilliseconds,11} |
{speedupFor,6:F2}x | {maxParallelFor,9} | {hilosParallelFor.Count,5}");
        Console.WriteLine($"Task.Run      | {swTaskRun.ElapsedMilliseconds,11} | {speedupTask,6:F2}x |
{maxTaskRun,9} | {hilosTaskRun.Count,5}");
        Console.WriteLine("================================================");

        if (!Console.IsInputRedirected)
        {
            Console.WriteLine("\nPresione cualquier tecla para salir...");
            Console.ReadKey();
        }
    }

    /// <summary>
    /// Simula el procesamiento de un trabajo y registra cuántas tareas se ejecutan a la vez.
    /// </summary>
    static void ProcesarTarea(int id, string origen, int duracionMs)
    {
        int enEjecucion = Interlocked.Increment(ref concurrenciaActual);
        ActualizarMaximo(enEjecucion);

        Thread hilo = Thread.CurrentThread;
        Console.WriteLine($"[{origen}] Hola Mundo desde la tarea {id,2} | Hilo
#{hilo.ManagedThreadId,2} " +
                          $"| ThreadPool: {hilo.IsThreadPoolThread,-5} | Simultáneas: {enEjecucion}");
        Thread.Sleep(duracionMs);

        Interlocked.Decrement(ref concurrenciaActual);
    }

    static void ActualizarMaximo(int valor)
    {
        int actual;
        while (valor > (actual = Volatile.Read(ref concurrenciaMaxima)))
        {
            if (Interlocked.CompareExchange(ref concurrenciaMaxima, valor, actual) == actual)
            {
                break;
```

```
                }
            }
        }

        static void ReiniciarContadores()
        {
            concurrenciaActual = 0;
            concurrenciaMaxima = 0;
        }

        static void ImprimirEncabezado(string titulo)
        {
            Console.WriteLine("\n------------------------------------------------");
            Console.WriteLine($" {titulo}");
            Console.WriteLine("------------------------------------------------");
        }
    }
}
```

7. Resultados

Resultados obtenidos en el equipo de pruebas (12 procesadores lógicos, 24 tareas por fase, 300 ms por tarea):

Estrategia	Tiempo (ms)	Speedup	Concurrencia máx.	Hilos usados
Secuencial	7469	1.00x	1	1
Parallel.For	641	11.65x	12	13
Task.Run	622	12.01x	12	12

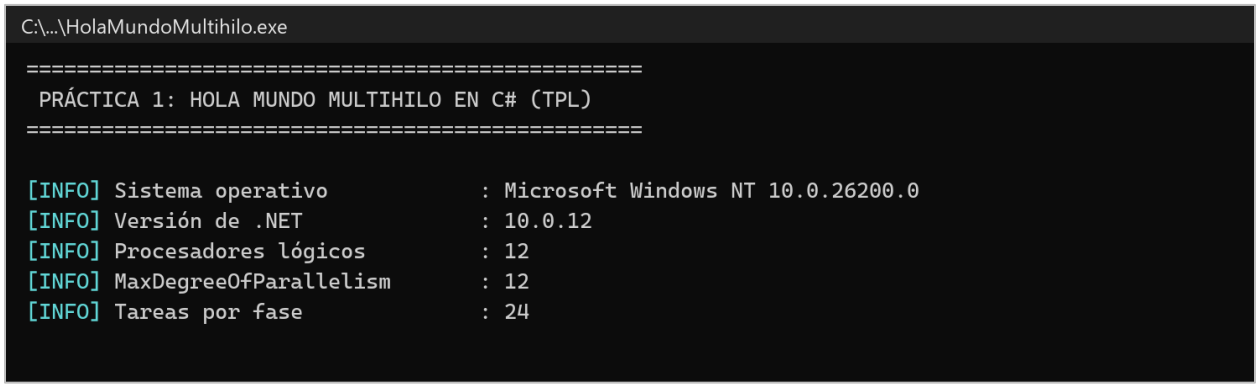


Figura 2. Detección de procesadores lógicos y configuración de MaxDegreeOfParallelism.

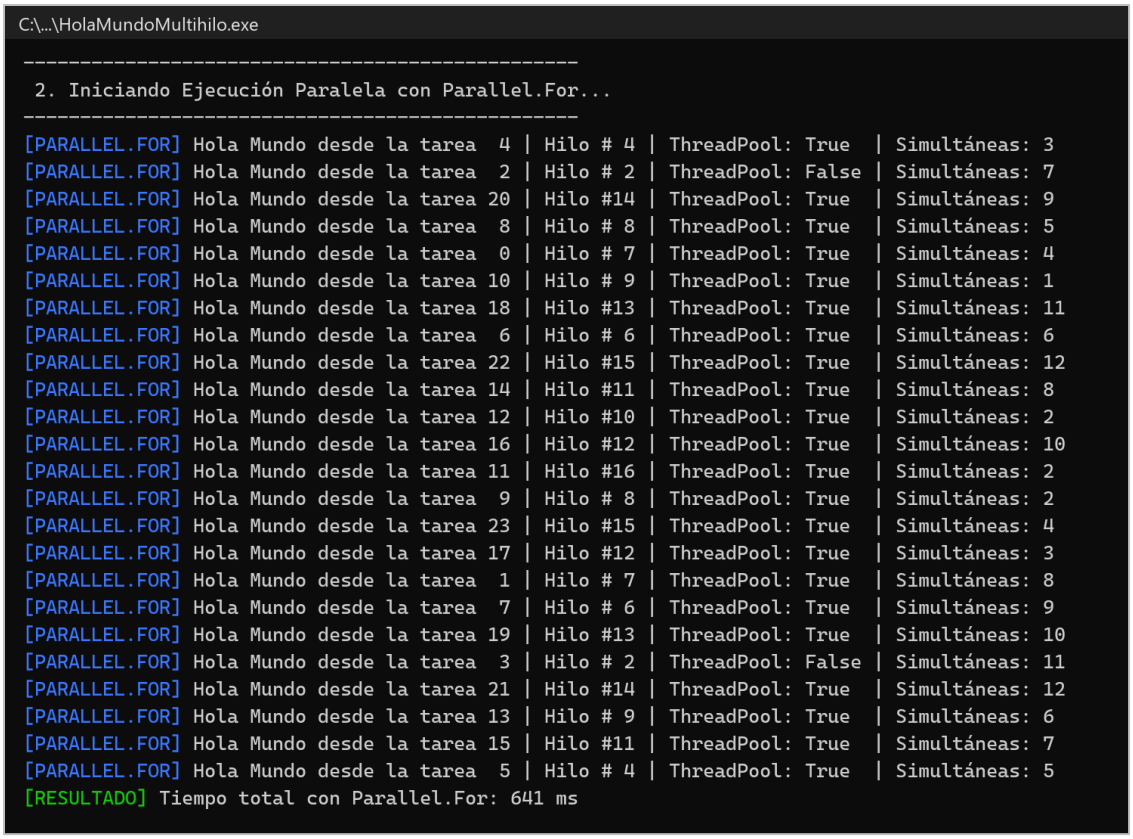


Figura 3. Ejecución de la fase Parallel.For: nunca hay más de 12 tareas simultáneas y los hilos se reutilizan.

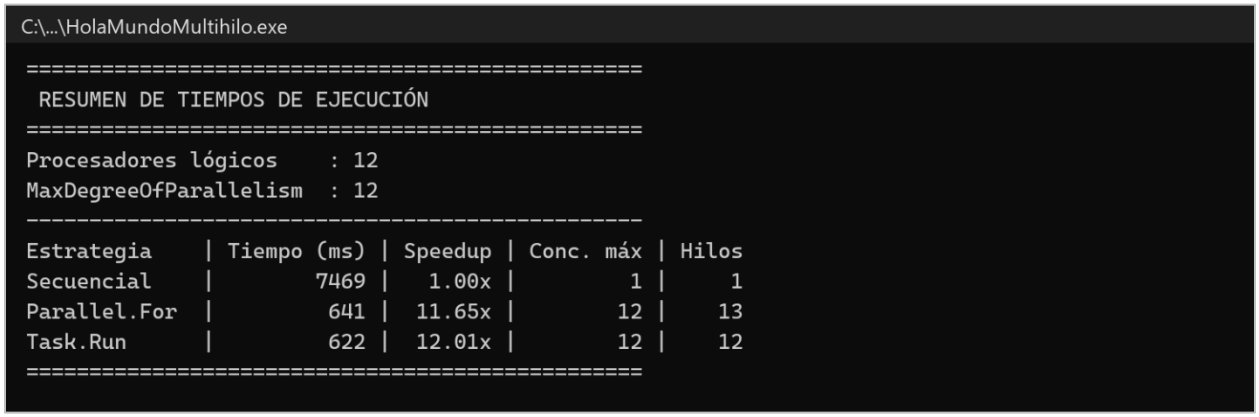


Figura 4. Resumen de tiempos de ejecución, speedup, concurrencia máxima e hilos utilizados.

7.1 Análisis

- **Tiempo:** la ejecución secuencial tardó unos 7.5 s (24 × 300 ms). Las versiones paralelas tardaron alrededor de 0.6 s, que corresponde a dos «rondas» de 12 tareas simultáneas (2 × 300 ms) más una pequeña sobrecarga de planificación.
- **Speedup:** se obtuvo un speedup cercano a 12, que es el máximo teórico para 12 procesadores lógicos.

- **Límite respetado:** la concurrencia máxima observada fue exactamente 12 en ambas fases paralelas, igual a `MaxDegreeOfParallelism`. Nunca se ejecutaron más tareas de las permitidas.
- **Reutilización de hilos:** la salida muestra que un mismo hilo ejecuta varias tareas (por ejemplo, el hilo #12 ejecutó las tareas 16 y 17). Esto confirma que la TPL reutiliza los hilos del ThreadPool en lugar de crear un hilo por tarea o fijar un hilo a cada núcleo. En la fase Parallel.For se usaron 13 hilos distintos aunque el límite era 12: la concurrencia nunca pasó de 12, pero el planificador fue cambiando qué hilos del pool atendían las tareas a lo largo de la ejecución.
- **Orden no determinista:** en las fases paralelas las tareas no terminan en orden (0, 1, 2…), porque el planificador decide cuándo se ejecuta cada una. Cada ejecución del programa puede mostrar un orden distinto.

8. Conclusiones

Se configuró correctamente el entorno de desarrollo con Visual Studio 2026 y .NET 10, y se desarrolló una aplicación de consola que adapta su nivel de paralelismo al hardware mediante `Environment.ProcessorCount`.

La práctica demostró que definir `MaxDegreeOfParallelism` a partir del número de procesadores lógicos permite aprovechar todo el procesador sin sobrecargarlo: se alcanzó un speedup de aproximadamente 12 veces y el límite de 12 tareas simultáneas se respetó en todo momento.

También se comprobó que no es necesario ni recomendable asignar manualmente un hilo a cada núcleo. La TPL y el ThreadPool gestionan dinámicamente los hilos, los reutilizan y distribuyen el trabajo de forma eficiente; el programador solo debe definir cuánto paralelismo desea como máximo.

Cabe señalar que la carga simulada (Thread.Sleep) no consume CPU, por lo que el speedup obtenido es ideal. Con cálculos intensivos reales el speedup suele ser algo menor debido a la sobrecarga de coordinación y a que los núcleos lógicos de Hyper-Threading comparten recursos físicos.

9. Repositorio

El código fuente completo de la práctica se encuentra disponible en: https://github.com/Printbriandev/Practica-1-p-paralela

10. Referencias

Albahari, J. (2023). C# 12 in a Nutshell: The Definitive Reference. O'Reilly Media. Capítulo 2: Building C# Applications; Capítulo 14: Concurrency and Threading.

Cleary, S. (2019). Concurrency in C# Cookbook (2.ª ed.). O'Reilly Media. Capítulo 1: Concurrency: An Overview; Capítulo 9: Testing Concurrent Code.

Microsoft. (s. f.). Task Parallel Library (TPL). Microsoft Learn. https://learn.microsoft.com/dotnet/standard/parallel-programming/task-parallel-library-tpl

Microsoft. (s. f.). ParallelOptions.MaxDegreeOfParallelism Property. Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.threading.tasks.paralleloptions.maxdegreeofparallelism

Microsoft. (s. f.). Environment.ProcessorCount Property. Microsoft Learn. https://learn.microsoft.com/dotnet/api/system.environment.processorcount